

Read-Copy-Update (RCU)

What is RCU?

RCU (Read-Copy-Update) is a synchronization mechanism that allows **multiple readers to access shared data without blocking**, while writers safely update the data.

It is mainly used in **read-heavy and write-light** workloads.

Why Do We Need RCU?

Consider a shared data structure:

Routing Table
Linked List
Hash Table
Process List

Access pattern:

Read Operations = Very Frequent

Write Operations = Rare

If we use a mutex or spinlock:

- Readers block each other.
- Readers wait for writers.
- Performance decreases.

RCU removes this overhead for readers.

RCU Principle

RCU follows the principle:

Readers should never block.

Instead of making readers wait, **writers do the extra work.**

How RCU Works

RCU consists of three steps:

Read
↓
Copy
↓
Update

Step 1: Read

Readers access the shared data.

Reader1

Reader2

Reader3

Features:

- No lock contention
- No busy waiting
- Very fast

Step 2: Copy

Instead of modifying the existing object,
the writer:

- Allocates a new object.
- Copies the old data.
- Makes changes in the new object.

Example:

Old Node

Value = 10

↓

New Node

Value = 20

The old object remains unchanged.

Step 3: Update

The writer atomically changes the pointer.

Before:

Global Pointer

|
V

Old Node

After:

Global Pointer



New Node

New readers see the new object.

Existing readers continue using the old object.

Why Not Free the Old Object Immediately?

Suppose:

Reader1 -----> Old Node

Writer -----> New Node

If the writer executes:

```
kfree(old_node);
```

Reader1 crashes because it is still accessing the old memory.

Grace Period

The writer waits until all existing readers finish reading.

This waiting time is called the:

Grace Period

After the grace period:

No reader is using old memory.

Now it is safe to free:

```
kfree(old_node);
```

Quiescent State

A CPU reaches a **Quiescent State** when it:

- Performs a context switch
- Goes to sleep
- Becomes idle

Once every CPU passes through a quiescent state,
the grace period ends.

Reader APIs

Header File:

```
#include <linux/rcupdate.h>
```

Enter RCU read section:

```
rcu_read_lock();
```

Read shared pointer:

```
rcu_dereference(ptr);
```

Exit RCU read section:

```
rcu_read_unlock();
```

Writer APIs

Assign new pointer:

```
rcu_assign_pointer(global_ptr, new_ptr);
```

Wait for readers:

```
synchronize_rcu();
```

Free old object:

```
kfree(old_ptr);
```

Complete Writer Flow

Allocate New Object

|

Copy Old Data

|

Modify New Object

|

```
rcu_assign_pointer()
```

|

```
synchronize_rcu()
```

|

```
kfree(Old Object)
```

Advantages

- Lock-free readers

- Extremely fast read performance
- Excellent scalability on multicore systems
- Readers never block writers
- Ideal for read-mostly workloads

Disadvantages

- Complex implementation
- Writers are slower
- Extra memory is needed temporarily
- Not suitable for write-intensive workloads

When to Use RCU?

Use RCU when:

- Read operations are much more frequent than writes.
- High read performance is required.
- Shared data is pointer-based.

Examples:

- Routing tables
- Kernel linked lists
- File descriptor tables
- Network configuration
- Process lists

When NOT to Use RCU?

Do not use RCU when:

- Writes are frequent.
- The shared data is a simple counter or flag.
- Simpler synchronization (mutex, spinlock, atomic operations) is sufficient.

RCU vs RW Locks

RCU	RW Locks
Readers are lock-free	Readers acquire read lock
Readers never block	Readers may wait for writers
Writers create a copy before updating	Writers modify data directly
Best for read-heavy workloads	Good for moderate read-heavy workloads
More complex	Simpler

Important Points

1. Readers never modify shared data.
2. Writers never modify the existing object directly.
3. Writers always create a new copy.
4. The pointer update is atomic.
5. Old memory is freed only after the grace period.
6. Readers execute with almost no synchronization overhead.

What does RCU stand for?

Read-Copy-Update.

Why is RCU faster than RW Locks?

Because readers do not acquire locks. They simply read the shared data, making the read path extremely fast.

What is a Grace Period?

The time during which all existing RCU readers finish before the old data can be safely freed.

Why doesn't the writer modify the existing object?

Because readers may still be using it. Modifying it directly could result in inconsistent or corrupted data.

Quick Revision

RCU (Read-Copy-Update)

Read



Copy



Update Pointer



Grace Period



Free Old Memory

Reader APIs

`rcu_read_lock()`

`rcu_dereference()`

`rcu_read_unlock()`

Writer APIs

`rcu_assign_pointer()`

`synchronize_rcu()`

`kfree()`

Advantages

✓ Lock-free readers

✓ Extremely fast reads

✓ Highly scalable

Disadvantages

✗ Complex

✗ Slower writers

✗ Extra temporary memory

```
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
vboxuser@ubuntu2204:~/synchronizaation/RCU_LOCKS$ ls
Makefile      Module.symvers rcu_demo.ko    rcu_demo.mod.c rcu_demo.o
modules.order rcu_demo.c      rcu_demo.mod  rcu_demo.mod.o README.md
vboxuser@ubuntu2204:~/synchronizaation/RCU_LOCKS$ sudo insmod rcu_demo.ko
[sudo] password for vboxuser:
vboxuser@ubuntu2204:~/synchronizaation/RCU_LOCKS$ sudo dmesg | tail -15
[ 9640.338848] Reader : value = 0
[ 9641.362275] Reader : value = 0
[ 9642.385986] Reader : value = 0
[ 9643.410888] Reader : value = 0
[ 9643.420134] Writer : updating value to 1
[ 9644.434435] Reader : value = 1
[ 9645.458132] Reader : value = 1
[ 9646.483354] Reader : value = 1
[ 9647.507177] Reader : value = 1
[ 9648.532470] Writer : updating value to 2
[ 9648.532500] Reader : value = 2
[ 9649.555467] Reader : value = 2
[ 9650.580905] Reader : value = 2
[ 9651.604000] Reader : value = 2
[ 9652.628731] Reader : value = 2
vboxuser@ubuntu2204:~/synchronizaation/RCU_LOCKS$
```